BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

It is impossible to program a mathematically perfect table driven agent as the size of the table that has to be stored will increase proportionally with the agents lifetime meaning for a game like chess where there is an estimated 10^120 game sequences the number of mappings would be so large that no human would be able to write each mapping , no storage device would reasonably be able to hold that many mappings and running a compiler for each mapping would be obsolete.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

```
if percept['wall_ahead'] or percept['toxin_ahead'] or percept['opponent_ahead']:
    return turn_left[percept['facing']]

if percept['food_ahead']:
    return percept['facing']`
```

The agent uses a group of simple deterministic IF THEN condition statements to perform an action based on only its current percepts with no reference or memory to its past percepts. For example if it sees a wall or an opponent or a toxin in front of it in the direction it is facing, it turns left but if it sees food or does not see anything it continues.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

Because the agent always makes the decision based on its current percepts alone. If it goes into a u shaped box with the exit not being accessible by just turning the same way, it will have no way of knowing that as it cannot see the direct locations of the walls, therefore it cannot reasonably determine where to go (partial observability) and it also does not have a relative internal map, so it cannot even trace back where it already was to not go that way again (no percept history to solve by elimination). Therefore the agent will keep going in an infinite loop.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Transition Model - The ModelBasedAgent keeps a rel_pos variable as its internal estimate of where it is relative to its starting point. This only updates when the previous action actually resulted in forward movement, checked via self.last_action == self.last_facing, since turning in place doesn't change position. When a forward move happens, dir_vectors[self.last_facing] gives a coordinate delta that gets added to rel_pos. The agent keeps track of how it moves in the world. However it's access to how the world evolves outside of itself is pretty limited as this is a partially observable environment. (Eg: cannot track opponent positions).

Sensor Model - get_percept() only returns local, immediate information: what's ahead, left, right, and at the current cell (wall, food, toxin, opponent), plus current facing. It's a single snapshot with no built-in history. This gets combined with the estimated rel_pos into state = (rel_pos, facing), stored in the visits dictionary, fusing sensed information with the transition estimate into something the agent can remember. This shows up in the wall-avoidance logic: if the same state has been visited twice or more, the agent turns right instead of left, breaking out of loops that would trap a SimpleReflexAgent forever. The agent's actions also affect the world in two distinct ways. A turn action only updates self.facing, position stays the same. A forward action (same facing as before) is what actually moves agent_pos, and it's the only case where rel_pos gets updated. The agent has to tell these apart to keep its internal tracking accurate, otherwise it'd drift out of sync with its real position.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING